

NUR Hand-in 2

Berend Nieuwhof

March 17, 2025

Abstract

In this document, the solutions to the second hand-in exercise are presented.

1 Exercise 1: Satellite galaxies

In this section, the solutions to the first exercise are presented.

Our script is given by:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import copy
4
5 def n(x, A, Nsat, a, b, c):
6     return A*Nsat*((x/b)**(a-3))*np.exp(-(x/b)**c)
7
8 def N(x, A, Nsat, a, b, c):
9     return n(x, A, Nsat, a, b, c)*4*np.pi*x**2
10
11 def romberg_interval(func, a, b, order, *args):
12     '''Calculates the integral of a function from a to b, using
13     some order of Richardson extrapolation'''
14     r = np.zeros(order)
15
16     # calculate r0
17     h = (b - a)
18     r[0] = 0.5 * h * (func(a, *args) - func(b, *args))
19     Np = 1
20     for i in np.arange(1, order-1): # calculate approximations
21         r[i] = 0
22         delta = h
23         h = 0.5 * h
24         x = a + h
25         for n in range(Np):
26             r[i] += func(x, *args)
27             x += delta
28         r[i] = 0.5*(r[i-1] + delta*r[i])
29         Np = 2*Np
30     Np = 1
31     for i in np.arange(1, order-1): # combine approximations. r[0] holds the best one
32         Np = 4*Np
33         for j in np.arange(0, order-i):
34             r[j] = (Np * r[j+1] - r[j]) / (Np - 1)
35
36     return r
37
38 def romberg(func, x, order, *args):
39     '''with x a linspace. calculates the integral over a range given by x.
40     len(x) is the number of function evaluations we use to integrate.'''
41     result = 0
42     for i in range(len(x)-1):
43         a = x[i]
44         b = x[i+1]
45         result += romberg_interval(func, a, b, order, *args)[0]
46     return result
```

```

47
48 def xor_shift(seed, a1 = 21, a2 = 35, a3 = 4):
49     '''Performs 64bit xor shift on the seed number.'''
50     state = np.uint64(seed)
51     a1 = np.uint64(a1)
52     a2 = np.uint64(a2)
53     a3 = np.uint64(a3)
54
55     state = state ^ (state >> a1)
56     state = state ^ (state << a2)
57     state = state ^ (state >> a3)
58     return state
59
60 def mult_w_carry(seed, a = 4294957665):
61     '''Performs multiply with carry, base 2**32'''
62     state = np.uint64(seed)
63     a = np.uint64(a)
64
65     state = a*(state & np.uint64(2**32 - 1)) + (state >> np.uint64(32))
66     number = state & np.uint64(2**32 - 1)
67
68     return number, state
69
70 def rng_int(seed, size):
71     state = np.uint64(seed)
72     result = np.zeros(size)
73     for i in range(size):
74         xor_state = xor_shift(state)
75         if xor_state < 2**32:
76             number, mult_state = mult_w_carry(xor_state)
77         else:
78             number, mult_state = mult_w_carry(xor_state & np.uint64(2**32 - 1))
79         state = mult_state
80         result[i] = number
81
82     return result
83
84 def rng_float(seed, size, a, b):
85     ints = rng_int(seed, size)
86     floats = a + (b-a)*(ints / (2**32))
87     return floats
88
89 def to_cartesian(r, theta, phi):
90     x = r * np.sin(phi) * np.cos(theta)
91     y = r * np.sin(phi) * np.sin(theta)
92     z = r * np.cos(phi)
93     return x, y, z
94
95 def mergesort(a):
96     N = len(a)
97     idx = np.arange(0, N, 1)
98     def mergesort_helper(idx):
99         if len(idx) > 1:
100             mid_idx = len(idx)//2
101             left = copy.deepcopy(idx[:mid_idx])
102             right = copy.deepcopy(idx[mid_idx:])
103
104             # recursively sort the left and right arrays
105             mergesort_helper(left)
106             mergesort_helper(right)
107             i, j, k = 0, 0, 0
108             # go through the arrays and check which element of is the smallest between
109             them, then add that one to a
110             while i < len(left) and j < len(right):
111                 if a[left[i]] < a[right[j]]:
112                     idx[k] = left[i]
113                     i += 1
114                 else:
115                     idx[k] = right[j]
116                     j += 1

```

```

116         k += 1
117         # if left or right still hold elements after one of the lists is ran through
        , put them into a in order
118         while i < len(left):
119             idx[k] = left[i]
120             i += 1
121             k += 1
122         while j < len(right):
123             idx[k] = right[j]
124             j += 1
125             k += 1
126
127     mergesort_helper(idx)
128     return idx
129
130 def n_deriv(x, A, Nsat, a, b, c):
131     return A*Nsat/b * ((a-3)*(x/b)**(a-4) - c*(x/b)**(a+c-4))*np.exp(-(x/b)**c)
132
133 def central_diff(x, func, h, *args):
134     deriv = (func(x + h, *args) - func(x - h, *args)) / (2*h)
135     return deriv
136
137 def ridder(x, func, h, d, *args, order = 5, tol = 1e-8):
138     r = np.zeros(order)
139     oneDeeth = 1 / d
140     r[0] = central_diff(x, func, h, *args)
141     for o in range(1, order):
142         h = h * oneDeeth
143         r[o] = central_diff(x, func, h, *args)
144     # now iterate to combine the previous
145
146     for j in range(1, order):
147         for i in range(order-1-j):
148             r[i] = (d**(2*j) * r[i+1] - r[i]) / (d**(2*j) - 1)
149             # TODO: Make sure Ridder stops at the right order.
150     return r[0]
151
152 A=1. # to be computed
153 Nsat=100
154 a=2.4
155 b=0.25
156 c=1.6
157
158 # We want to integrate the function (with A = 1) to find what we need to set A to
159 # to find <Nsat> = 100. A will be 100 divided by the value of the integral.
160
161 # Because n(x) is radially symmetric, we can use the fact that
162 # dV = 4pi*r**2 dr to write down the integral.
163
164 x = np.linspace(0.00001, 5, 15)
165 integ_result = romberg(N, x, 10, A, 100, a, b, c)
166 new_A = Nsat / integ_result
167 print('A found by integral normalisation:', new_A)
168
169 check = romberg(N, x, 10, new_A, Nsat, a, b, c)
170 check = Nsat / check
171 print('Sanity check:', check)
172 #Plot of histogram in log-log space with line (question 1b)
173 xmin, xmax = 10**-4, 5
174 N_generate = 10000
175
176 # First sample a uniform distribution for r, theta and phi
177 Pu1 = rng_float(42, N_generate, 0, 1)
178 Pu2 = rng_float(41, N_generate, 0, 1)
179
180 theta = np.pi*(1 - 2*Pu1)
181 phi = 2*np.pi * Pu2
182
183 # Make the analytical function
184 relative_radius = np.linspace(xmin, xmax, 10000)

```

```

185 analytical_function = N(relative_radius, new_A, Nsat, a, b, c) / Nsat
186
187 # find the maximum value by sorting
188 sorting_idx2 = mergesort(analytical_function)
189 sorted_analytical = analytical_function[sorting_idx2]
190 max_val2 = sorted_analytical[-1]
191
192 # Do rejection sampling for r.
193 Pu3 = 10**rng_float(44, N_generate*10, np.log10(xmin), np.log10(xmax))
194 Pu4 = rng_float(33, N_generate*10, 0, max_val2)
195 P_x = N(Pu3, new_A, Nsat, a, b, c) / Nsat
196
197 rej_result = []
198 for i in range(N_generate*10):
199     if Pu4[i] < P_x[i]:
200         rej_result.append(Pu3[i])
201         if len(rej_result) == 10000:
202             break
203 rej_result = np.array(rej_result) # take the first 10000 elements
204
205 #21 edges of 20 bins in log-space
206 edges = 10*np.linspace(np.log10(xmin), np.log10(xmax), 21)
207 hist = np.histogram(rej_result, bins=edges)[0]
208 hist_scaled = Nsat * hist / (len(Pu3)) # normalize by the amount of samples originally
    used.
209
210 hist_px = np.histogram(P_x, bins = edges)[0]
211 hist_px_scaled = Nsat * hist / (len(P_x))
212
213 fig1b, ax = plt.subplots()
214 # ax.stairs(hist_px_scaled, edges = edges, fill=False, label = 'calculated Px')
215 ax.stairs(hist_scaled, edges=edges, fill=False, label='Satellite galaxies')
216 plt.plot(relative_radius, analytical_function, 'r-', label='Analytical solution')
217 ax.set(xlim=(xmin, xmax), ylim=(10*(-3), 10), yscale='log', xscale='log',
218        xlabel='Relative radius', ylabel='Number of galaxies')
219 ax.legend()
220 plt.savefig('my_solution_1b.png', dpi=600)
221 plt.close()
222
223 #Cumulative plot of the chosen galaxies (1c)
224 # We'll achieve this by generating a random sequence using our random number generator.
225 # We will then sort the array and thus generate a shuffle index which we then apply to
226 # the samples from above.
227 rnd_seq = rng_float(69, 10000, 0, 1)
228 rnd_shuffle_idx = mergesort(rnd_seq)
229
230 # Select 100 random satellite galaxies by applying the index to the samples
231 # and then selecting the first 100
232 chosen = rej_result[rnd_shuffle_idx][:100]
233 sorting_idx = mergesort(chosen)
234 chosen_sorted = chosen[sorting_idx]
235
236 fig1c, ax = plt.subplots()
237 ax.plot(chosen_sorted, np.arange(100))
238 ax.set(xscale='log', xlabel='Relative radius',
239        ylabel='Cumulative number of galaxies',
240        xlim=(xmin, xmax), ylim=(0, 100))
241 plt.savefig('my_solution_1c.png', dpi=600)
242
243 # Now we will calculate dn(x)/dx at x = 1.
244 x = 1
245 h = 0.2
246
247 deriv_x = ridder(x, n, h, 2, new_A, Nsat, a, b, c, order = 10)
248 deriv_x_a = n_deriv(x, new_A, Nsat, a, b, c)
249 print('derivative of n with respect to x')
250 print("with Ridder's method:", deriv_x)
251 print('Analytically:', deriv_x_a)

```

Satellites.py

1.1 1a. Finding A by numerical integration.

In order to find the normalization constant A in the formula given in the problem sheet, we implemented a Romberg integration algorithm. For the integral, we use $dV = 4\pi r^2 dr$. For the rest of this exercise, $n(x)$ will refer to the number density, and $N(x)$ will be the number density multiplied by $4\pi r^2$. The relevant code is given below:

```
1 def n(x, A, Nsat, a, b, c):
2     return A*Nsat*((x/b)**(a-3))*np.exp(-(x/b)**c)
3
4 def N(x, A, Nsat, a, b, c):
5     return n(x, A, Nsat, a, b, c)*4*np.pi*x**2
6
7 def romberg_interval(func, a, b, order, *args):
8     '''Calculates the integral of a function from a to b, using
9     some order of Richardson extrapolation'''
10    r = np.zeros(order)
11
12    # calculate r0
13    h = (b - a)
14    r[0] = 0.5 * h * (func(a, *args) - func(b, *args))
15    Np = 1
16    for i in np.arange(1, order-1): # calculate approximations
17        r[i] = 0
18        delta = h
19        h = 0.5 * h
20        x = a + h
21        for n in range(Np):
22            r[i] += func(x, *args)
23            x += delta
24        r[i] = 0.5*(r[i-1] + delta*r[i])
25        Np = 2*Np
26    Np = 1
27    for i in np.arange(1, order-1): # combine approximations. r[0] holds the best one
28        Np = 4*Np
29        for j in np.arange(0, order-i):
30            r[j] = (Np * r[j+1] - r[j]) / (Np - 1)
31
32    return r
33
34 def romberg(func, x, order, *args):
35     '''with x a linspace. calculates the integral over a range given by x.
36     len(x) is the number of function evaluations we use to integrate.'''
37     result = 0
38     for i in range(len(x)-1):
39         a = x[i]
40         b = x[i+1]
41         result += romberg_interval(func, a, b, order, *args)[0]
42     return result
43
44 def xor_shift(seed, a1 = 21, a2 = 35, a3 = 4):
45     '''Performs 64bit xor shift on the seed number.'''
46     state = np.uint64(seed)
47     a1 = np.uint64(a1)
48     a2 = np.uint64(a2)
49     a3 = np.uint64(a3)
50
51     state = state ^ (state >> a1)
52     state = state ^ (state << a2)
53     state = state ^ (state >> a3)
54     return state
55
56 def mult_w_carry(seed, a = 4294957665):
57     '''Performs multiply with carry, base 2**32'''
58     state = np.uint64(seed)
59     a = np.uint64(a)
60
61     state = a*(state & np.uint64(2**32 - 1)) + (state >> np.uint64(32))
62     number = state & np.uint64(2**32 - 1)
63
```

As can be seen in the code above, romberg_interval calculates the integral between a and b up to some order, and romberg then sums this over many intervals. In this case, we go up to order 10, where we use 15 function evaluations. The output given by this piece of the code is given below. For the sanity check, we calculate the integral again with our calculated value of A, and we see that it is equal to one.

```
1 A found by integral normalisation: 9.211900153463198
2 Sanity check: 1.0000000000000007
```

satellite_output.txt

1.2 1b. Generating satellite positions

For this subquestion, we made use of rejection sampling in log space to sample 10,000 points following the analytical distribution $N(x)dx/\langle N_{sat} \rangle$. The relevant code is given below.

```
1
2 def rng_int(seed, size):
3     state = np.uint64(seed)
4     result = np.zeros(size)
5     for i in range(size):
6         xor_state = xor_shift(state)
7         if xor_state < 2**32:
8             number, mult_state = mult_w_carry(xor_state)
9         else:
10            number, mult_state = mult_w_carry(xor_state & np.uint64(2**32 - 1))
11            state = mult_state
12            result[i] = number
13
14     return result
15
16 def rng_float(seed, size, a, b):
17     ints = rng_int(seed, size)
18     floats = a + (b-a)*(ints / (2**32))
19     return floats
20
21 def to_cartesian(r, theta, phi):
22     x = r * np.sin(phi) * np.cos(theta)
23     y = r * np.sin(phi) * np.sin(theta)
24     z = r * np.cos(phi)
25     return x, y, z
26
27 def mergesort(a):
28     N = len(a)
29     idx = np.arange(0, N, 1)
30     def mergesort_helper(idx):
31         if len(idx) > 1:
32             mid_idx = len(idx)//2
33             left = copy.deepcopy(idx[:mid_idx])
34             right = copy.deepcopy(idx[mid_idx:])
35
36             # recursively sort the left and right arrays
37             mergesort_helper(left)
38             mergesort_helper(right)
39             i, j, k = 0, 0, 0
40             # go through the arrays and check which element of is the smallest between
41             them, then add that one to a
42             while i < len(left) and j < len(right):
43                 if a[left[i]] < a[right[j]]:
44                     idx[k] = left[i]
45                     i += 1
46                 else:
47                     idx[k] = right[j]
48                     j += 1
49                 k += 1
```

```

49         # if left or right still hold elements after one of the lists is ran through
        , put them into a in order
50         while i < len(left):
51             idx[k] = left[i]
52             i += 1
53             k += 1
54         while j < len(right):
55             idx[k] = right[j]
56             j += 1
57             k += 1
58
59     mergesort_helper(idx)
60     return idx
61
62 def n_deriv(x, A, Nsat, a, b, c):
63     return A*Nsat/b * ((a-3)*(x/b)**(a-4) - c*(x/b)**(a+c-4))*np.exp(-(x/b)**c)
64
65 def central_diff(x, func, h, *args):
66     deriv = (func(x + h, *args) - func(x - h, *args)) / (2*h)
67     return deriv
68
69 def ridder(x, func, h, d, *args, order = 5, tol = 1e-8):
70     r = np.zeros(order)
71     oneDeeth = 1 / d
72     r[0] = central_diff(x, func, h, *args)
73     for o in range(1, order):
74         h = h * oneDeeth
75         r[o] = central_diff(x, func, h, *args)
76     # now iterate to combine the previous
77
78     for j in range(1, order):
79         for i in range(order-1-j):
80             r[i] = (d**(2*j) * r[i+1] - r[i]) / (d**(2*j) - 1)
81             # TODO: Make sure Ridder stops at the right order.
82     return r[0]
83
84 A=1. # to be computed
85 Nsat=100
86 a=2.4
87 b=0.25
88 c=1.6
89
90 # We want to integrate the function (with A = 1) to find what we need to set A to
91 # to find <Nsat> = 100. A will be 100 divided by the value of the integral.
92
93 # Because n(x) is radially symmetric, we can use the fact that
94 # dV = 4pi*r**2 dr to write down the integral.
95
96 x = np.linspace(0.00001, 5, 15)
97 integ_result = romberg(N, x, 10, A, 100, a, b, c)
98 new_A = Nsat / integ_result
99 print('A found by integral normalisation:', new_A)
100
101 check = romberg(N, x, 10, new_A, Nsat, a, b, c)
102 check = Nsat / check
103 print('Sanity check:', check)
104 #Plot of histogram in log-log space with line (question 1b)
105 xmin, xmax = 10**-4, 5
106 N_generate = 10000
107
108 # First sample a uniform distribution for r, theta and phi
109 Pu1 = rng_float(42, N_generate, 0, 1)
110 Pu2 = rng_float(41, N_generate, 0, 1)
111
112 theta = np.pi*(1 - 2*Pu1)
113 phi = 2*np.pi * Pu2
114
115 # Make the analytical function
116 relative_radius = np.linspace(xmin, xmax, 10000)
117 analytical_function = N(relative_radius, new_A, Nsat, a, b, c) / Nsat

```

```

118
119 # find the maximum value by sorting
120 sorting_idx2 = mergesort(analytical_function)

```

Satellites.py

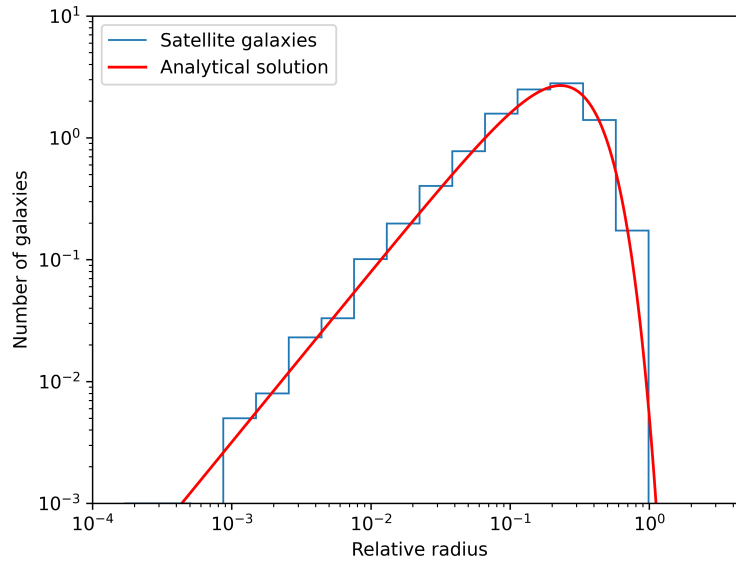
The analytical function is equal to $N(x) = n(x)4\pi r^2$. We first find the maximum this function takes on in our interval, by evaluating the function at many points (which is already necessary for plotting), sorting the array of evaluations, and finding the first element of it. We use this maximum as the maximum for our uniform random numbers.

Speaking of uniform random numbers, we generate ours as follows: we feed a number into a `xor_shift` algorithm, and the result of that is put into a Multiply With Carry random number generator. The MWC returns both a number and a state: the number is output as a random number, and the state is then fed back into the `xor_shift` algorithm, where the next random number will be generated. This of course generates integers between 0 and 2^{32} , so we use the `rng_float` function to generate uniform random floats between two values.

We use rejection sampling to sample the distribution, and we do so by sampling our x value in log space, to match the logarithmic bins given by the exercise instructions. We then compare another random number $y = U(0, \max)$, with $\max$ being the maximum calculated earlier, to the function evaluated at our x value. If y is smaller than x , we accept y as part of our sample. We generate samples until we've reached 10,000, at which point we stop generating them.

The result of this sampling is plotted below. Note that we normalize our histogram by dividing by the total amount of samples generated to reach 10,000 accepted samples, and multiplying by the average number of satellites.

Figure 1: In red, the analytical distribution that we are supposed to sample from is plotted. The blue bars correspond to the logarithmic bins given, and their heights are the number of samples per bin. The bars follow the distribution nicely.



1.3 1c. Random galaxy selection

In this subquestion, we randomly select satellite galaxies from the sample in b in a way that selects every galaxy with equal probability, does not draw the same galaxy twice, and does not reject any draw. The relevant code is included here. All sorting in this exercise is done with `mergesort`, which can be found in the first code snippet in this exercise.

```

1 sorted_analytical = analytical_function[sorting_idx2]
2 max_val2 = sorted_analytical[-1]
3
4 # Do rejection sampling for r.
5 Pu3 = 10**rng_float(44, N_generate*10, np.log10(xmin), np.log10(xmax))
6 Pu4 = rng_float(33, N_generate*10, 0, max_val2)
7 P_x = N(Pu3, new_A, Nsat, a, b, c) / Nsat
8
9 rej_result = []
10 for i in range(N_generate*10):
11     if Pu4[i] < P_x[i]:
12         rej_result.append(Pu3[i])
13         if len(rej_result) == 10000:
14             break
15 rej_result = np.array(rej_result) # take the first 10000 elements
16
17 #21 edges of 20 bins in log-space
18 edges = 10**np.linspace(np.log10(xmin), np.log10(xmax), 21)
19 hist = np.histogram(rej_result, bins=edges)[0]

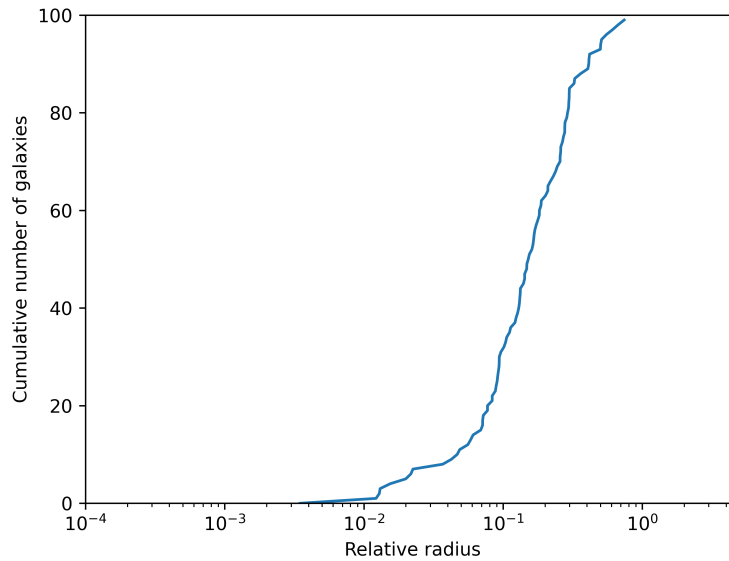
```

Satellites.py

We achieve this by generating an array of 10,000 random numbers, and sorting them to create a random sorting index. We then apply this sorting index to the random sample from b , and select the first 100 elements. This achieves the requirements specified above.

Then, we sort the 100 galaxies from smallest to largest radius using our implementation of mergesort, and plot the number of galaxies within a radius.

Figure 2: We see that the number of galaxies within a radius is strictly increasing, which means the array is sorted correctly.



1.4 1d. Derivative of $n(x)$

In this section, we calculate the derivative $dn(x)/dx$ at $x = 1$ numerically. We used Ridder's algorithm to find the derivative, as seen in the code below.

```

1 hist_px = np.histogram(P_x, bins = edges)[0]
2 hist_px_scaled = Nsat * hist / (len(P_x))
3
4

```

```

5 fig1b, ax = plt.subplots()
6 # ax.stairs(hist_px_scaled, edges = edges, fill=False, label = 'calculated Px')
7 ax.stairs(hist_scaled, edges=edges, fill=False, label='Satellite galaxies')
8 plt.plot(relative_radius, analytical_function, 'r-', label='Analytical solution')
9 ax.set(xlim=(xmin, xmax), ylim=(10**(-3), 10), yscale='log', xscale='log',
10        xlabel='Relative radius', ylabel='Number of galaxies')
11 ax.legend()
12 plt.savefig('my_solution_1b.png', dpi=600)
13 plt.close()
14
15 #Cumulative plot of the chosen galaxies (1c)
16 # We'll achieve this by generating a random sequence using our random number generator.
17 # We will then sort the array and thus generate a shuffle index which we then apply to
18 # the samples from above.
19 rnd_seq = rng.float(69, 10000, 0, 1)
20 rnd_shuffle_idx = mergesort(rnd_seq)
21
22 # Select 100 random satellite galaxies by applying the index to the samples
23 # and then selecting the first 100
24 chosen = rej_result[rnd_shuffle_idx][:100]
25 sorting_idx = mergesort(chosen)
26 chosen_sorted = chosen[sorting_idx]
27
28 fig1c, ax = plt.subplots()
29 ax.plot(chosen_sorted, np.arange(100))
30 ax.set(xscale='log', xlabel='Relative radius',
31        ylabel='Cumulative number of galaxies',
32        xlim=(xmin, xmax), ylim=(0, 100))

```

Satellites.py

The output is given here:

```

1 derivative of n with respect to x
2 with Ridder's method: -0.6264877290898574
3 Analytically: -0.6264877290898926

```

satellite_output.txt

2 Exercise 2: Vandermonde Matrix

In this section we look at exercise 2

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import copy
4 import timeit
5
6 def bisection(func, brac, *args, abs_tol = 1e-8, frac_tol = 1e-8, max_iter = 50):
7     '''implements the bisection method of root finding.'''
8     i = 0
9     a = brac[0]
10    b = brac[1]
11    # check if a, b is a bracket
12    # assert func(a, *args)*func(b, *args) < 0
13
14    while i < max_iter:
15        # Divide the bracket in 2
16        c = (a+b)/2
17        if func(a, *args)*func(c, *args) < 0:
18            b = c
19        else:
20            a = c
21        i += 1
22        if abs(a-b) < abs_tol or abs((a-b)/a) < frac_tol:
23            return a, i
24    return a, i
25

```

```

26 def secant(func, guesses, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
27     '''implements the secant method for root finding.'''
28     i = 2
29     # r will hold all guesses
30     r = np.zeros(max_iter)
31     r[0] = guesses[0]
32     r[1] = guesses[1]
33     while i < max_iter:
34         r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
35         * func(r[i-1], *args)
36         if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
37             return r[i], i
38             i += 1
39
40     print('maximum secant iterations reached')
41     return r[i-1], i
42
43 def false_position(func, brac, *args, abs_tol = 1e-10, frac_tol = 1e-10, max_iter = 50):
44     '''implements the false position method for root finding.'''
45     i = 2
46     r = np.zeros(max_iter)
47     r[0] = brac[0]
48     r[1] = brac[1]
49     # check if a, b is a bracket
50     assert func(r[0], *args)*func(r[1], *args) < 0
51
52     while i < max_iter:
53         r[i] = r[i-1] - ((r[i-1] - r[i-2])/(func(r[i-1], *args) - func(r[i-2], *args)))
54         * func(r[i-1], *args)
55         if func(r[i], *args)*func(r[i-2], *args) < 0:
56             r[i-1] = r[i-2]
57             if abs(r[i] - r[i-1]) < abs_tol or abs((r[i] - r[i-1])/r[i]) < frac_tol:
58                 return r[i], i
59             i += 1
60
61     return r[i-1], i
62
63 ## 2a.
64
65 k=1.38e-16 # erg/K
66 aB = 2e-13 # cm^3 / s
67 Z = 0.015 # unitless
68 psi = 0.929 # unitless
69 Tc = 1e4 # K
70
71 # here no need for nH nor ne as they cancel out
72 def equilibrium1(T,Z,Tc,psi):
73     return psi*Tc*k - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T*k
74
75 # test and visualize first
76 temps = np.linspace(1, 1e7, 1000)
77 eqs = equilibrium1(temps, Z, Tc, psi)
78 # plt.plot(temps, eqs)
79 # plt.show()
80
81 brac1 = [1, 1e7] # K
82 print(brac1)
83 midpoint1 = 0.5e7 # mid point of the bracket interval
84 print('-----2a-----')
85
86 root1, num_it1 = bisection(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
87 print('root and number of iterations')
88 print('using bisection:', root1, num_it1)
89 print('function evaluated at root:', equilibrium1(root1, Z, Tc, psi))
90
91 root2, num_it2 = secant(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01, frac_tol = 1e-8, max_iter = 50)
92 print('using secant method:', root2, num_it2)
93 print('function evaluated at root:', equilibrium1(root2, Z, Tc, psi))

```

```

92 root3, num_it3 = false_position(equilibrium1, brac1, Z, Tc, psi, abs_tol = 0.01,
93     frac_tol = 1e-8, max_iter = 50)
94 print('using false position method:', root3, num_it3)
95 print('function evaluated at root:', equilibrium1(root3, Z, Tc, psi))
96
97 def equilibrium2(T,Z,Tc,psi, nH, A, xi):
98     return (psi*Tc - (0.684 - 0.0416 * np.log(T/(1e4 * Z*Z)))*T - .54 * ( T/1e4 )**.37 *
99         T)*k*nH*aB + A*xi + 8.9e-26 * (T/1e4)
100
101 print('—————2b—————')
102 # 2b
103 A = 5e-10 # erg
104 xi = 1e-15 # s**-1
105 brac2 = [1, 1e15] # K
106 k=1.38e-16 # erg/K
107 aB = 2e-13 # cm^3 / s
108 Z = 0.015 # unitless
109 psi = 0.929 # unitless
110 Tc = 1e4 # K
111
112 # test and visualize first
113 temps = np.linspace(1, 1e15, 1000)
114 ne = 1e5
115 eqs = equilibrium2(temps, Z, Tc, psi, ne, A, xi)
116 # plt.plot(temps, eqs)
117 # plt.show()
118
119 for ne in [1e-4, 1, 1e4]:
120     print('\nfor ne = ', ne)
121     root1, num_it1 = bisection(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e
122         -10, frac_tol = 1e-10, max_iter = 100)
123     print('root and number of iterations')
124     print('using bisection:', root1, num_it1)
125     print('function evaluated at root:', equilibrium2(root1, Z, Tc, psi, ne, A, xi))
126
127     root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
128         frac_tol = 1e-10, max_iter = 100)
129     print('using secant method:', root2, num_it2)
130     print('function evaluated at root:', equilibrium2(root2, Z, Tc, psi, ne, A, xi))
131
132     root3, num_it3 = false_position(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol
133         = 1e-10, frac_tol = 1e-10, max_iter = 100000)
134     print('using false position method:', root3, num_it3)
135     print('function evaluated at root:', equilibrium2(root3, Z, Tc, psi, ne, A, xi))
136
137 print('—————')
138 def smallest_ne():
139     ne = 1e-4
140     root1, num_it1 = bisection(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e
141         -10, frac_tol = 1e-10, max_iter = 100)
142     return root1, num_it1
143
144 def middle_ne():
145     ne = 1
146     root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
147         frac_tol = 1e-10, max_iter = 100)
148     return root2, num_it2
149
150 def big_ne():
151     ne = 1e4
152     root2, num_it2 = secant(equilibrium2, brac2, Z, Tc, psi, ne, A, xi, abs_tol = 1e-10,
153         frac_tol = 1e-10, max_iter = 100)
154     return root2, num_it2
155
156 small_ne_time = timeit.timeit(lambda : smallest_ne(), number = 1)
157 mid_ne_time = timeit.timeit(lambda : middle_ne(), number = 1)
158 big_ne_time = timeit.timeit(lambda : big_ne(), number = 1)
159
160 print('Bisection for smallest n took:', small_ne_time)

```

```
154 | print('Secant for middle ne took:', mid_ne_time)
155 | print('Secant for biggest ne took:', big_ne_time)
```

HII-regions.py